

LINQ OfType Operator in C# with Examples

In this article, I will discuss the **LINQ OfType Operator in C#** with Examples. Please read our previous article before proceeding to this article, where we discussed the **Where Filtering Operator in C#** with Examples. The OfType Operator belongs to the filtering category of LINQ operators. As part of this article, we will discuss the following pointers in detail.

1. **What is the OfType Operator in LINQ?**
2. **Examples using both Method and Query syntax.**
3. **Difference between OfType and is Operator in C#.NET.**
4. **When to Use OfType Operator in LINQ?**

What is the OfType Operator in LINQ?

The LINQ OfType Operator in C# filters specific data from a data source based on the data type we passed to this operator. For example, if we have a collection that stores integer and string values, and if we need to fetch either the integer values or only the string values from that collection, then we need to use the **LINQ OfType Operator**.

Following is the signature of the LINQ OfType Method. It is implemented as a generic type. So, there is no overloaded version available for this method. This method can take any data type and then fetch the specified values from the collection.

```
public static IEnumerable<TResult> OfType<TResult>(this IEnumerable source);
```

The OfType operator in LINQ filters a sequence, returning only those elements that can be cast to a specified type. This is useful when dealing with a non-generic IEnumerable or a collection of objects of various types. Here's how it works:

- It filters the elements based on their ability to be cast to a specific type.
- It excludes elements that cannot be cast to that type rather than throwing an InvalidCastException.

How Does the OfType Operator Works?

- **Filtering Based on Type:** The OfType operator examines each element in the source collection. It checks whether each element is of a specified type or a derived type.
- **Type Casting:** If an element is of the specified type or a derived type, OfType includes it in the result. It implicitly casts the element to the specified type.
- **Ignoring Non-Matching Elements:** Elements that are not of the specified type are simply ignored and not included in the result.
- **Returning a Filtered Collection:** The operator produces an IEnumerable<T> where T is the specified type. This means the resulting collection contains elements only of the type specified, and they are already cast to that type.

Example to Understand LINQ OfType Operator

Let's say we have a collection of object type. The Object class is the superclass of all data types to store values like the one below.

```
List<object> dataSource = new List<object>()
{
    "Tom", "Mary", 50, "Prince", "Jack", 10, 20, 30, 40, "James"
};
```

Our requirement is to fetch all the integer values from the collection by ignoring the string values. We can achieve this easily by using the LINQ OfType Method in C# as follows.

```
List<object> dataSource = new List<object>()
{
    "Tom", "Mary", 50, "Prince", "Jack", 10, 20, 30, 40, "James"
};
```

```
List<int> intData = dataSource.Of<int>().ToList();
```

50 10 20 30 40

Of<int> returns List<int>

Example to Understand LINQ OfType Method Using Method Syntax in C#:

In the below example, the dataSource collection contains both integer and string data. Then, using the OfType<int>() method, we fetched only the integer data from the data source using the LINQ Method syntax.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source Contains both Integer and String Data
            List<object> dataSource = new List<object>()
            {
                "Tom", "Mary", 50, "Prince", "Jack", 10, 20, 30, 40, "James"
            };
            //Fetching only the Integer Data from the Data Source
            //using Linq Method Syntax and OfType Method
            List<int> intData = dataSource.Of<int>().ToList();
            //Print the Integer Data
            foreach (int number in intData)
            {
                Console.Write(number + " ");
            }
            Console.ReadKey();
        }
    }
}
```

```

    }
}
}

```

Output: 50 10 20 30 40

Example to Understand LINQ OfType Operator Using Query Syntax in C#:

We can achieve this using two ways, i.e., using the OfType Method and IS Operator. In the following example, the collection contains both string and integer values, and we are only fetching the string values using both the OfType method and IS Operator with the LINQ Query syntax.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source Contains both Integer and String Data
            List<object> dataSource = new List<object>()
            {
                "Tom", "Mary", 50, "Prince", "Jack", 10, 20, 30, 40, "James"
            };
            //Fetching only the Integer Data from the Data Source
            //using Linq Query Syntax and OfType Method
            var intData = (from num in dataSource.OfType<int>()
                           select num).ToList();
            //Print the Integer Data
            Console.WriteLine("Using OfType Method");
            foreach (int number in intData)
            {
                Console.Write(number + " ");
            }
            Console.WriteLine("\nUsing IS Operator");
            //Fetching only the String Data from the Data Source
            //using Linq Query Syntax and is Method
            var stringData = (from name in dataSource
                              where name is string
                              select name).ToList();
            //Print the Integer Data
            foreach (string name in stringData)
            {
                Console.Write(name + " ");
            }
            Console.ReadKey();
        }
    }
}

```

```
}
```

Output:

```
Using OfType Method
50 10 20 30 40
Using IS Operator
Tom Mary Prince Jack James
```

OfType and IS Operators with a Condition in C#:

Let's say we want to retrieve all the names with lengths of more than 3 characters and all the integer numbers greater than 30. Here, we will use the OfType Method to retrieve all the integers greater than 30 and the "IS" operator to retrieve all the strings with lengths greater than 3 characters.

```
using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            List<object> dataSource = new List<object>()
            {
                "Tom", "Mary", 50, "Prince", "Jack", 10, 20, 30, 40, "James"
            };
            //Fetching the Integer Numbers which are greater than 30
            //Using Method Syntax
            var intData = dataSource.OfType<int>().Where(num => num > 30).ToList();
            foreach (int number in intData)
            {
                Console.Write(number + " ");
            }
            Console.WriteLine();
            //Fetching the String Names whose length is greater than 3 characters
            //Using Query Syntax with is Operator
            var stringData = (from name in dataSource
                             where name is string && name.ToString().Length > 3
                             select name).ToList();
            //Fetching the String Names whose length is greater than 3 characters
            //Using Query Syntax with OfType Operator
            var stringData2 = (from name in dataSource.OfType<string>()
                              where name.Length > 3
                              select name).ToList();
            foreach (string name in stringData2)
            {
                Console.Write(name + " ");
            }
        }
    }
}
```

```

    }
    Console.ReadKey();
}
}
}

```

Output:

```

50 40
Mary Prince Jack James

```

OfType Operator in C# with Where Condition:

Let us see how we can use the LINQ OfType Operator in C# with Method and Query Syntax with the Where Condition. In the below example, we are retrieving all the numbers greater than 30 using Method and Query Syntax using the OfType operator.

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQOfTypeDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Data Source which contains both String and Integer Data
            List<object> dataSource = new List<object>()
            {
                "Tom", "Mary", 50, "Prince", "Jack", 10, 20, 30, 40, "James"
            };
            //Fetching the Integer numbers from the Data Source where the Number
            //Greater than 30
            //Using Method Syntax
            var intData = dataSource.OfType<int>().Where(num => num > 30).ToList();
            foreach (int number in intData)
            {
                Console.Write(number + " ");
            }
            Console.WriteLine();
            //Using Query Syntax with OfType Operator
            var intData1 = (from num in dataSource.OfType<int>()
                           where num > 30
                           select num).ToList();
            //Using Query Syntax with is Operator
            //Here, we need to type cast num to int before applying the > operator
            var intData2 = (from num in dataSource
                           where num is int && (int)num > 30
                           select num).ToList();

```

```

        foreach (int name in intData2)
        {
            Console.Write(name + " ");
        }
        Console.ReadKey();
    }
}

```

Output:

```

50 40
50 40

```

Note: In LINQ, the Where Extension Method filters the data source based on a condition, i.e., predicate function. The OfType Extension Method filters the data source based on a given type. We can use both Where and OfType Methods multiple times in a single LINQ Query.

When to Use OfType Operator in LINQ?

The OfType operator in LINQ is best used when dealing with collections of objects of various types. You want to filter the collection so that only elements of a particular type are included. Here are some scenarios where the OfType operator is particularly useful:

Working with Heterogeneous Collections:

When you have a collection that stores objects of various types, such as `object[]` or `List<object>`, you must operate on elements of a particular type within that collection. Filtering an `ArrayList` that contains a mix of `int`, `string`, and other objects to retrieve only the `int` objects.

```

var mixedCollection = new List<object> { 1, "two", 3.0, "four", 5 };
var stringsOnly = mixedCollection.OfType<string>();

```

Filtering by Type:

When you want to filter elements based on their type before applying further operations like filtering, projecting, or aggregating, this is especially helpful in situations where not all elements in the collection are of the desired type.

```

var animals = new List<Animal> { new Dog(), new Cat(), new Dog(), new Bird() };
var dogsOnly = animals.OfType<Dog>();

```

Dealing with Mixed Data:

In scenarios where data can have mixed types due to various data sources or dynamic data, you can use `OfType` to extract and work with elements of a specific type.

When processing data from sources like JSON, XML, or a database, you might get a mixed-type result set,

and you need to filter by type before processing. Selecting only numeric types from a mixed collection for statistical analysis.

```
var mixedData = GetDataFromMultipleSources();  
var specificData = mixedData.OfType<DataType>();
```

Avoiding Invalid Casts:

To avoid invalid cast exceptions, especially when dealing with elements that may not be safely castable to the desired type, you can use `OfType` to ensure that only compatible elements are included. Safely selecting elements of a certain type from a mixed collection without causing runtime exceptions.

```
var mixedData = GetMixedData();  
var safeData = mixedData.OfType<DataType>();
```

In the next article, I will discuss the [SET Operators in LINQ](#) with Examples. I hope this article gives you a very good understanding of the concept of **LINQ OfType Operator** in C# with Examples.